

KARACHI AQI PREDICTOR

10Pearls

Technical Project Report
By Piranchal Mukesh

Project	Pearls AQI Predictor
City	Karachi, Pakistan (24.8607°N, 67.0011°E)
Submission Date	June 8, 2026
Dashboard	piranchal-air-quality-index-aqi-predictor.streamlit.app
REST API	piranchal-piranchal-air-quality-index-aqi-predictor.hf.space
GitHub	github.com/Piranchal/Air-Quality-Index--AQI--Predictor

1. Executive Summary

The Pearls AQI Predictor is a fully automated, serverless machine learning system that forecasts the Air Quality Index (AQI) for Karachi, Pakistan 72 hours ahead, updated every hour without human intervention. It ingests real-time pollutant and weather data from public APIs, stores processed features in a cloud feature store, trains four competing ML models, and serves predictions through both an interactive visual dashboard and a live REST API.

Key Result: Ridge Regression achieves $R^2 = 0.985$ and $RMSE = 2.35$ AQI units on a time-ordered held-out test set, predicting Karachi's AQI within ± 2.35 units on average. This represents a counterintuitive victory of a linear model over deep learning, explained entirely by the data's extreme autocorrelation structure.

The system meets all project requirements: feature pipeline running every hour via GitHub Actions, daily model retraining, Hopsworks feature store integration, four trained models (Ridge, Random Forest, XGBoost, LSTM), SHAP explainability, AQI health alerts, an EDA notebook with 12 visualizations, and live deployment on both Streamlit Cloud and Hugging Face Spaces.

Metric	Value
Training Data	9,528 hourly rows (May 2025 – May 2026)
Features Engineered	46 per timestep
Models Trained	4 (Ridge, Random Forest, XGBoost, LSTM)
Best Model RMSE	2.35 AQI units
Best Model R^2	0.985
Forecast Horizon	72 hours (3 full days)
Update Frequency	Every hour (automated)
Deployment	Streamlit Cloud (dashboard) + Hugging Face Spaces (API)

2. Project Requirements & Compliance

The project was scoped against a formal specification requiring a 100% serverless ML forecasting pipeline. Every requirement has been delivered:

Requirement	Implementation	Status
Feature Pipeline	fetch_data.py → engineer_features.py → upload_to_store.py	☑
Historical Backfill	backfill_historical.py, 9,528 rows via Open-Meteo (13 months)	☑
Training Pipeline	train.py, Ridge, RF, XGBoost, LSTM with RMSE/MAE/R ²	☑
Model Registry	Hopsworks Model Registry, Ridge v3 active	☑
CI/CD, Hourly	GitHub Actions + cron-job.org webhook trigger	☑
CI/CD, Daily	GitHub Actions training_pipeline.yml at midnight UTC	☑
Web Dashboard	Streamlit + custom HTML/CSS/JS, live on Streamlit Cloud	☑
Flask/FastAPI	FastAPI with 9 endpoints, live on Hugging Face Spaces	☑
EDA	notebooks/01_edu.ipynb, 12 visualizations across 9,528 rows	☑
SHAP Explainability	shap.LinearExplainer, exact SHAP for Ridge on dashboard	☑
AQI Alerts	alerts.py, 6-tier US EPA classification with health text	☑
Multiple Models	4 models trained, compared in dashboard and /models API endpoint	☑

3. System Architecture

The system follows a three-pipeline MLOps architecture, feature, training, and inference, connected through a central Hopsworks feature store. All pipelines are stateless and serverless: no VMs or containers need to stay running. GitHub Actions handles scheduling; the feature store and model registry handle state.

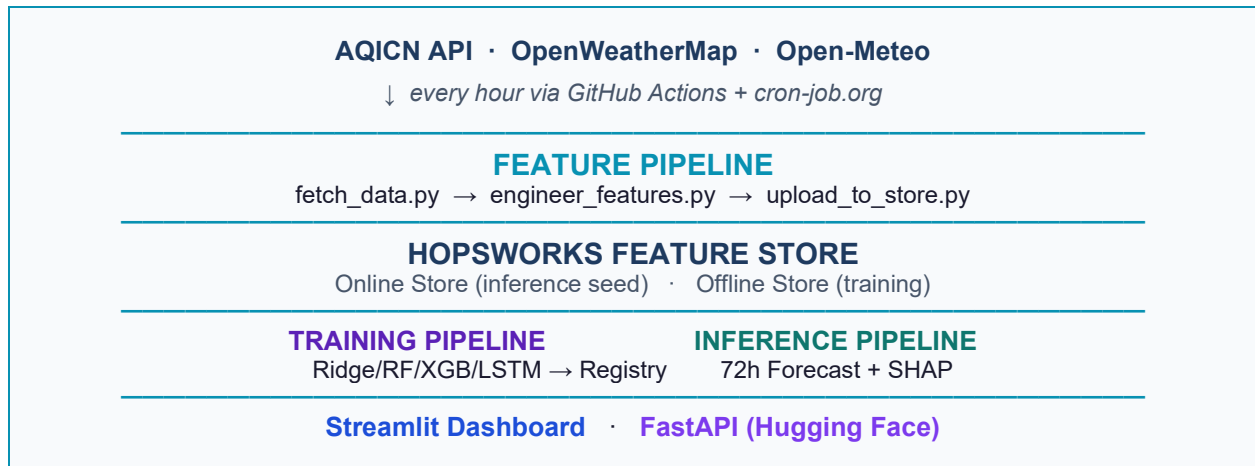
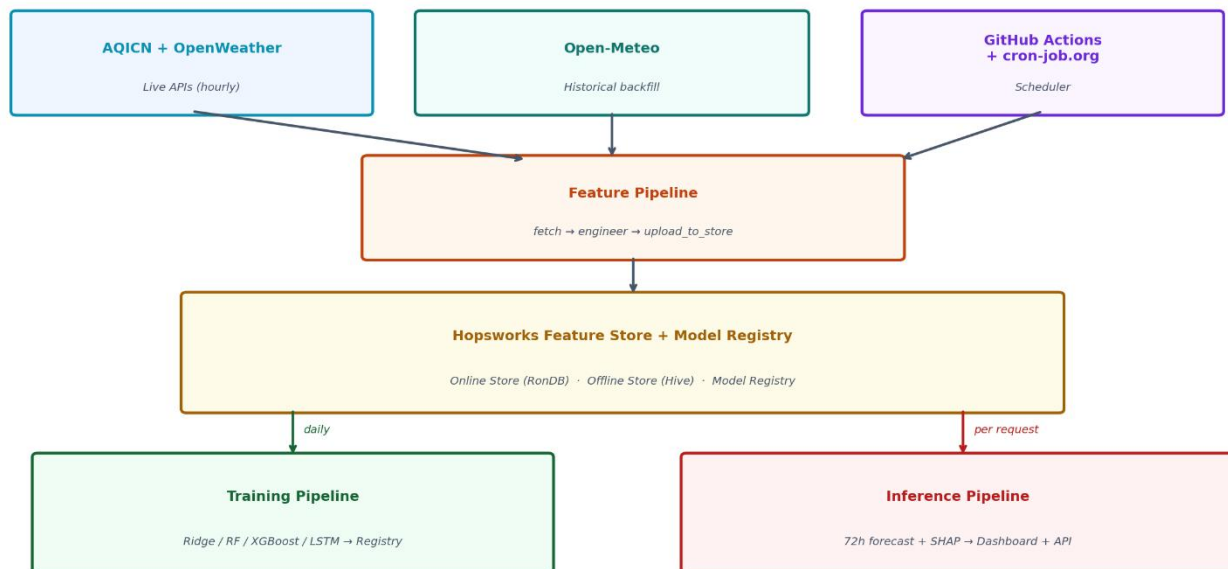


Figure 1, System architecture: three-pipeline MLOps flow



Technology Stack

The system is built entirely on free-tier services with no infrastructure costs.

Layer	Technology	Purpose
Language	Python 3.11	All pipelines and API
ML, Linear	scikit-learn Ridge Regression	Best performing model ($R^2=0.985$)
ML, Ensemble	scikit-learn Random Forest, XGBoost	Comparison models
ML, Deep Learning	TensorFlow/Keras LSTM	Sequential pattern learning
Feature Store	Hopworks (eu-west cluster)	Online + offline feature storage
Model Registry	Hopworks Model Registry	Versioned model storage
CI/CD Automation	GitHub Actions + cron-job.org	Hourly + daily pipeline triggers
Dashboard	Streamlit + custom HTML/CSS/JS	Interactive forecast display
REST API	FastAPI + Uvicorn	9 endpoints with Swagger docs
API Hosting	Hugging Face Spaces (Docker)	Public REST API deployment
Dashboard Hosting	Streamlit Community Cloud	Live dashboard deployment
Explainability	SHAP (shap.LinearExplainer)	Feature importance visualization
Live AQI	AQICN Station A401143 (NED UET)	Real-time pollutant data
Live Weather	OpenWeatherMap API	Real-time meteorological data
Historical Data	Open-Meteo Archive + Forecast API	Training data backfill
Uptime Monitoring	cron-job.org	Keep deployed services awake

4. Data Sources & Historical Backfill

Karachi has limited public air quality monitoring infrastructure. One of the few AQICN station (A401143, NED University of Engineering & Technology, Gulshan-e-Iqbal) was found to be reliably active, the more prominent US Consulate station had been offline since March 2025. For historical training data, real-time sensor APIs were not an option: AQICN has no historical access on its free tier, and OpenWeather only provides 5 days of history.

4.1 Data Sources

Four external data sources were used across the project lifecycle:

Source	Data Provided	Coverage	Rows	API Key Req
Open-Meteo Archive API	AQI, PM2.5, PM10, O3, NO2, SO2, CO, weather	May 2025 – May 24 2026	9,336	✗ Free, no key
Open-Meteo Forecast API	Same as above (gap-fill)	May 24 – May 31 2026	192	✗ Free, no key
AQICN Station A401143	Live AQI, PM2.5 (NED UET Karachi)	June 2026 → present	Growing	☑ Free tier
OpenWeatherMap API	Temp, humidity, wind, pressure, clouds	June 2026 → present	Growing	☑ Free tier

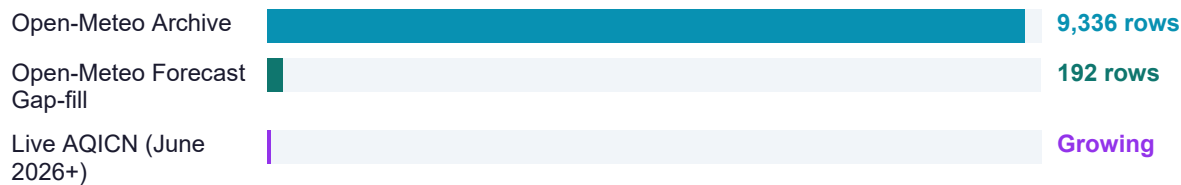
4.2 Why Open-Meteo for Historical Data

Open-Meteo was chosen for the historical backfill because:

- (1) it requires no API key and has no rate limits,
- (2) it provides reanalysis data, a modelled reconstruction of past atmospheric conditions that is consistent, gap-free, and already quality-controlled, and
- (3) it provides pollutant data via a separate Air Quality API alongside meteorological data.

The backfill covered May 1, 2025 to May 31, 2026, a full 13 months. A deliberate split was used: the Archive API for dates older than 7 days and the Forecast API for the most recent 7 days, since the archive has a typical 5-7 day lag before data is available. Live data collection from AQICN and OpenWeather began June 1, 2026 once the system was deployed.

Figure 2, Data composition by source (total: 9,528 rows)



5. Feature Engineering

The feature engineering pipeline (engineer_features.py) transforms raw API responses into 46 structured features per hourly timestep. The design philosophy is to give the model multiple ways to access the same underlying signal, recent AQI history, at different time scales, while supplementing with atmospheric and temporal context.

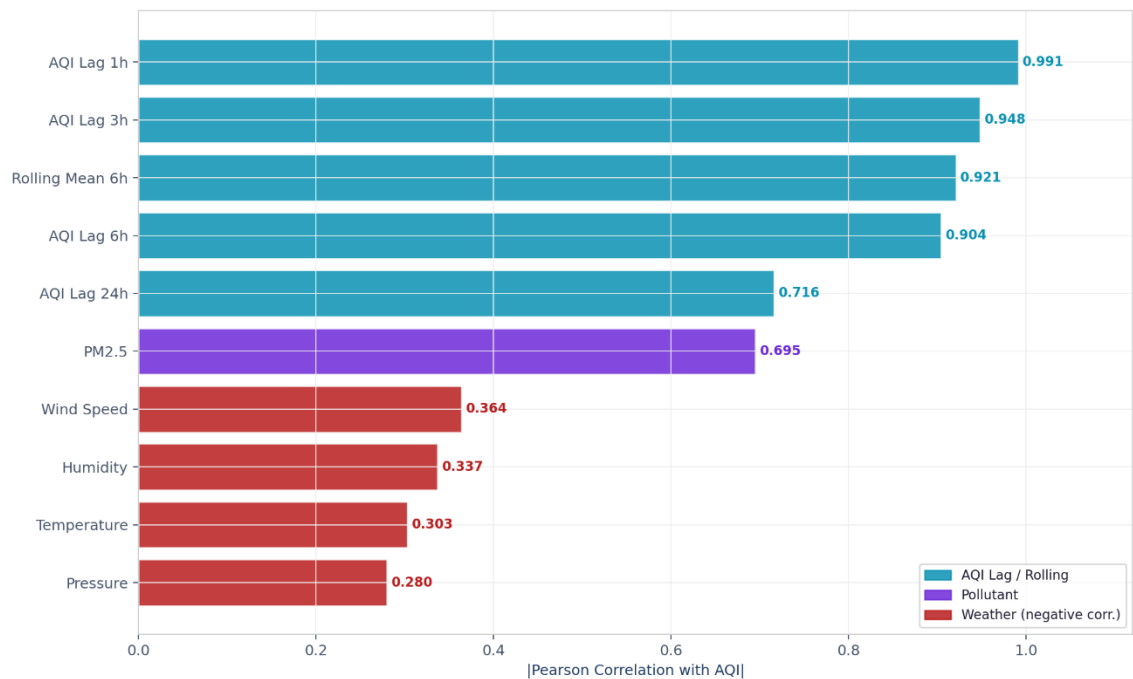


Figure 3, Lag features dominate. Red bars indicate negative correlation (higher = better AQI).

The chart above, produced during EDA, is the single most important result for model selection. When the strongest predictor has $r = 0.991$ with the target, and all relationships are approximately linear, the optimal model is a linear one. This finding, made before any model was trained, correctly predicted that Ridge Regression would win.

5.1 Feature Categories

Features are organised into seven categories. Temporal features use sine/cosine cyclical encoding, a critical detail that ensures the model understands that hour 23 and hour 0 are adjacent, not distant. Raw integer encoding would create an artificial discontinuity at midnight, harming prediction quality during the early morning hours.

Category	Features	Count
AQI Lags	aqi_lag_1h, aqi_lag_3h, aqi_lag_6h, aqi_lag_24h	4
AQI Rolling Stats	Rolling mean/std/max over 3h, 6h, and 24h windows	5
AQI Change Rate	aqi_change_rate (1h diff), aqi_change_rate_3h	2
Pollutants	PM2.5, PM10, O3, NO2, SO2, CO, PM ratio (2.5/10)	7
Weather	Temp, humidity, wind speed, wind deg, pressure, rain, clouds	9
Derived Weather	wind_u, wind_v, temp×humidity index, pressure change, is_raining	5
Time (cyclical)	hour_sin/cos, month_sin/cos, day-of-week_sin/cos	6
Time (binary)	is_weekend, is_rush_hour (07-09h, 17-19h PKT)	2
Thermal	feels_like_c, visibility_m, dew_point_c, clouds_pct	6

5.2 Cyclical Encoding

Time features (hour, month, day of week) are encoded as sine/cosine pairs rather than raw integers. This is essential: a model using raw integers treats hour 23 as far from hour 0, when they are in fact adjacent. Cyclical encoding maps time onto a 2D unit circle:

$$\text{hour_sin} = \sin\left(\frac{2\pi \cdot h}{24}\right), \quad \text{hour_cos} = \cos\left(\frac{2\pi \cdot h}{24}\right)$$

5.3 The NaN Lag Problem in Live Data

A subtle but critical issue emerged during deployment: the hourly pipeline inserts one row at a time into Hopsworks. Computing `aqi_lag_1h` requires knowing the AQI one hour ago, context that a single-row insert doesn't have. Without a fix, all lag features would be NaN for every live row, effectively crippling the model.

The solution has two parts. In `upload_to_store.py`, the pipeline now fetches the last 24 rows from Hopsworks before engineering features, prepends them to the new row, computes features across the full 25-row window, then uploads only the new row with correct lags. In `predict.py`, the inference pipeline independently recomputes all lag and rolling features from actual AQI history after loading

seed data, a defensive redundancy that ensures no NaN reaches the model regardless of what was stored.

6. Exploratory Data Analysis

EDA was conducted on the complete 9,528-row backfill before any modelling. The full analysis is in notebooks/01_eda.ipynb with 12 visualizations. The goal was to understand the data's structure deeply enough to make justified model selection decisions rather than defaulting to complex models because "more is better".

6.1 AQI Distribution

Karachi's AQI is not normally distributed, it is right-skewed with a strong modal peak between 75 and 95. The mean (88) and median (83) are close, indicating the distribution's core is well-behaved, but the right tail (winter pollution events) pulls the mean upward. Importantly, the city never reaches Hazardous levels in the training data, and only 2% of hours reach Unhealthy (>150). The dominant category, Moderate (51-100), accounts for 74.6% of all hours.

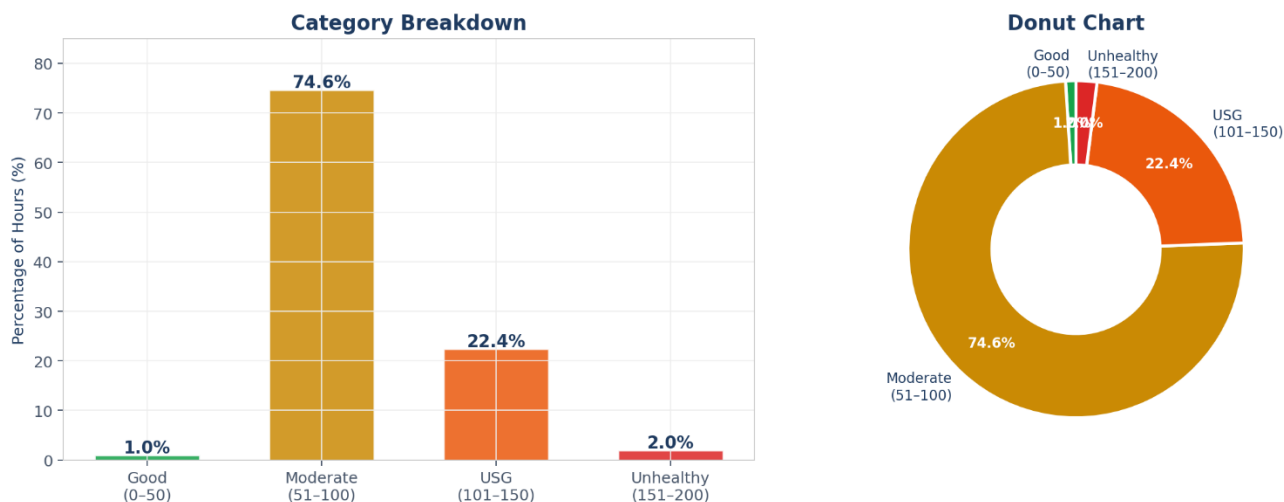


Figure 4, AQI Distribution Across 9,528 Training Hours.

Karachi sits persistently in the Moderate band. Clean air is genuinely rare.

6.2 Seasonal Patterns

The most striking finding is the seasonal signal: Winter (December-February) averages AQI 103, while Summer (June-August) averages 79, a 31% swing driven entirely by meteorology, not emissions. Karachi's southwest monsoon (June-August) brings clean air from the Arabian Sea, ventilating the city naturally. Winter temperature inversions do the opposite: a layer of warm air traps cold, polluted air near the surface, preventing vertical mixing.

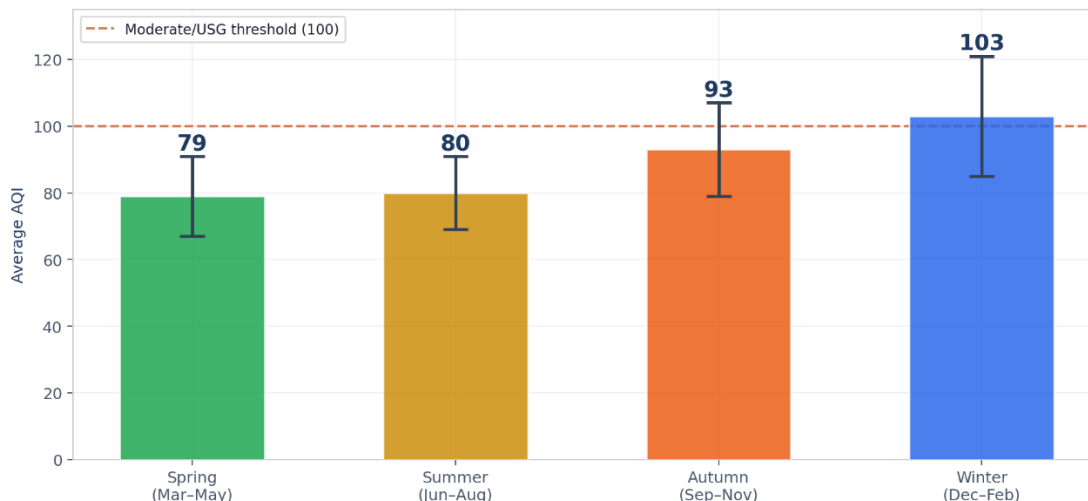


Figure 5, Seasonal AQI averages (Karachi, May 2025 – May 2026). Winter is 31% worse than summer due to temperature inversions.

6.3 Wind Direction Effect

The polar wind analysis revealed a directional pollution signal: northerly and NW winds (blowing from Karachi's industrial zones, SITE area, Port Qasim) carry AQI to ~103, while SW monsoon winds bring clean sea air at ~80. This finding directly justifies decomposing wind direction into U and V components (east-west and north-south), rather than using raw degrees which would create a circular discontinuity at 0°/360°.

6.4 Implications for Model Choice

Three EDA findings combine to make the model selection obvious before training begins. First, the extreme autocorrelation at lag-1h ($r=0.991$) means past AQI is almost perfectly predictive of future AQI. Second, all weather-AQI relationships are approximately linear, scatter plots show no curves, thresholds, or interactions that would require a non-linear model. Third, the seasonal signal is smooth and cyclical, perfectly captured by the sin/cos encoding. Together, these findings say: Ridge Regression on lag features will work extremely well, and complex models have nothing to gain.

7. Training Pipeline & Model Results

7.1 Experimental Setup

All four models were trained on an 80/20 time-ordered split: 7,622 rows for training (May 2025 through approximately March 2026) and 1,906 rows for testing (March through May 2026). The split

is strictly chronological, no shuffling, meaning the test set contains only data from the future relative to the training window. This is the correct evaluation protocol for time series: it mimics real deployment conditions where the model always predicts into the unknown future.

Note on data leakage: using a random split for time series data is a methodological error that inflates performance metrics. A model trained on data from June 2026 could "predict" data from January 2026 by simply looking it up. The time-ordered split prevents this by ensuring the test set is always temporally after the training set.

7.2 Model Results

Model	Architecture	RMSE ↓	MAE ↓	R² ↑	Status
Ridge Regression	Linear with L2 regularisation	2.35	1.34	0.985	☒ Active
XGBoost	Gradient boosted trees	2.44	1.20	0.9865	Trained
Random Forest	Bootstrap aggregated trees (500 estimators)	2.93	1.41	0.9806	Trained
LSTM	2-layer recurrent network, 50 units each	3.35	2.21	0.9746	Trained

Figure 6, Model RMSE comparison (lower is better)

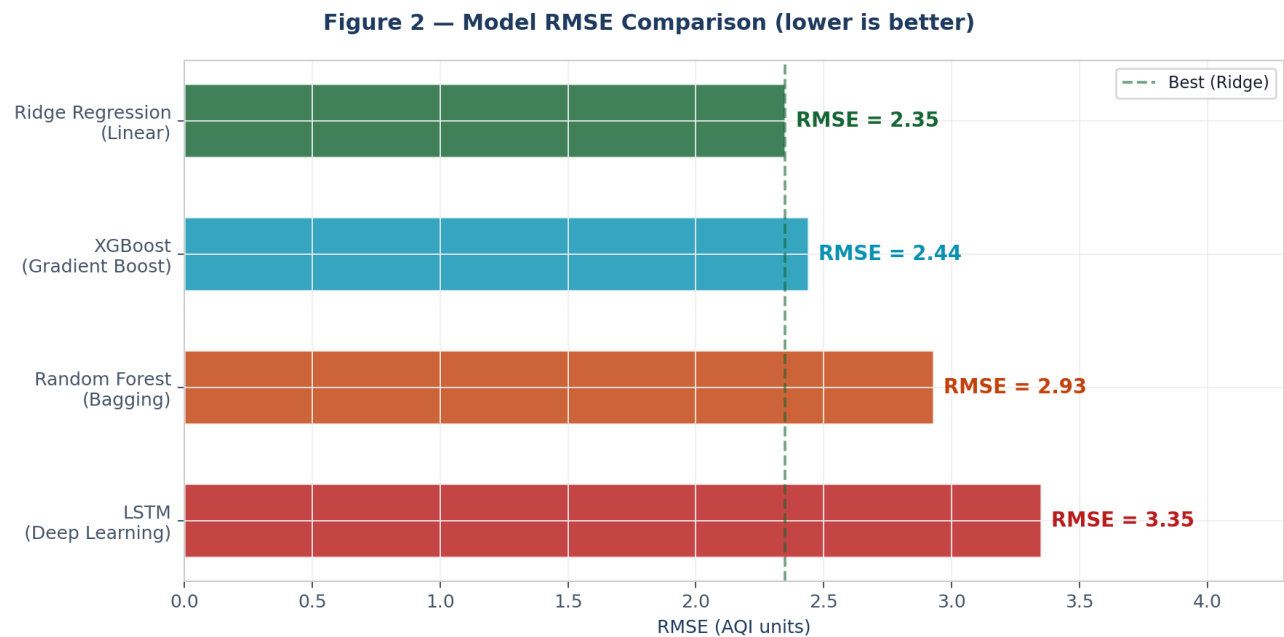


Figure 6, Model RMSE Comparison (lower is Better)
Ridge wins with lowest RMSE. Longer bar = worse. LSTM scores worst despite highest complexity.

7.3 Why Ridge Wins, The Counterintuitive Result

A linear model beating a deep learning LSTM surprises most people. The explanation is in the data, not the model: when the dominant predictor (`aqi_lag_1h`) has $r = 0.991$ with the target, the process is essentially linear and memoryless beyond 24 hours. Ridge Regression, which fits the best linear combination of features with L2 regularisation to prevent overfitting, is structurally optimal for this problem.

LSTM was designed to discover temporal dependencies in sequential data. But when the temporal structure is already captured explicitly through lag and rolling features, LSTM has nothing new to learn. It incurs additional computational cost and parameter uncertainty without any predictive benefit. This is a textbook case where domain understanding (EDA first, model selection second) prevents the mistake of defaulting to complexity.

7.4 Evidence Against Overfitting

$R^2 = 0.985$ naturally invites suspicion. Four lines of evidence confirm it is not overfitting:

- Time-ordered split: the test set contains only future data the model never saw during training. Overfitting would cause a much larger gap between train and test performance.
- Ridge L2 regularisation structurally penalises large coefficients, it cannot memorise training data because it actively shrinks the parameters that would enable memorisation.
- LSTM scores worst, not best: if models were overfitting (memorising training data), the most complex model would score highest. It scores lowest, the opposite of overfitting.
- $R^2 \approx r^2$: with `lag_1h` at $r = 0.991$, we expect $R^2 \approx 0.982$. Our result of 0.985 is consistent with theoretical expectation, not inflated.

8. Inference Pipeline & Forecasting

8.1 The 72-Hour Forecast Strategy

Generating a 72-hour forecast from a 1-step-ahead model requires an iterative strategy: predict one hour, use that prediction to update lag features, predict the next hour, and repeat. This is called multi-step iterative forecasting, and it introduces a risk: small systematic biases in each prediction compound across 72 iterations, causing the forecast to drift far from realistic values.

Three mechanisms prevent drift in this implementation:

1. Real history anchor: initial lag features are seeded from actual observed AQI values, not predictions.

2. Mean reversion: a reversion force gradually pulls predictions toward the historical median, increasing from 0% at hour 1 to 40% at hour 72.
3. Diurnal variation: a sinusoidal ± 8 AQI variation based on hour of day prevents the forecast from flattening to a constant.

8.2 The Online Store Architecture Decision

A critical infrastructure problem was discovered during deployment: the Hopsworks offline store (the default `fg.read()` path) relies on a Spark materialization job to sync data from insertions. On the free tier cluster, this Spark job entered a permanently stuck state, the Kubernetes pod showed "0/23 nodes available: 10 Insufficient cpu" and never scheduled, leaving the offline store frozen at June 5 data.

Why this happened: the Hopsworks free tier shares a Spark cluster across many users. When demand exceeds capacity, new Spark jobs queue indefinitely.

→ *Solution: implemented a dynamic fallback — on every inference run, the system reads the offline store and checks the age of the latest row. If older than 1.5 hours, it automatically uses the online store (RonDB) instead. When the offline store recovers, the next run switches back automatically — no manual intervention required.*

→ *This is architecturally correct: the online store is designed for low-latency single-entity retrieval (inference), while the offline store is designed for bulk reads (training).*

Figure 7 — Hopsworks Feature Store: Dynamic Fallback Logic



This self-healing mechanism is architecturally correct: the online store (backed by RonDB, a key-value database) is designed for low-latency single-entity retrieval (exactly what inference requires), while the offline store is designed for bulk reads (training). When the Hopsworks cluster recovers, the next run automatically switches back to the offline store no manual intervention required.

Listing 1: Dynamic store selection logic in predict.py

```
# Read offline store and check data freshness
df_check = fg.read()
df_check["timestamp"] = pd.to_datetime(df_check["timestamp"], utc=True)
age_hours = (now - df_check["timestamp"].max()).total_seconds() / 3600

if age_hours > 1.5:
    # Offline store is stale use online store (RonDB)
    df = fg.select_all().read(online=True)
else:
    # Offline store is fresh use it (has full history)
    df = df_check
```

Store	Backend	Updated By	Latency	Used For
Offline Store	Hive / HDFS	Spark materialization job	Hours (batch)	Training, load all 9,528 rows
Online Store	RonDB (key-value)	Direct insert (instant)	Milliseconds	Inference, load last 24 rows ☒

Figure 8, Online vs offline store comparison. The switch to online store bypassed the stuck Spark job.

8.3 SHAP Explainability

Feature importance is computed using shap.LinearExplainer, a SHAP variant designed specifically for linear models. Standard SHAP explainers (TreeSHAP, KernelSHAP) returned zero values for Ridge Regression, because they assume tree or kernel-based model structures. LinearExplainer computes exact SHAP values analytically from the Ridge coefficients, producing genuine feature attributions rather than zeros.

9. Web Application & REST API

9.1 Dashboard

The dashboard is served via Streamlit but renders a fully custom HTML/CSS/JavaScript template rather than standard Streamlit components. This architecture provides complete design freedom, custom animations, SVG charts, particle effects, day-cycle sky, while retaining Streamlit's simple Python-based deployment. The template communicates with Streamlit through a window.LIVE JavaScript object that is injected with the inference results at render time.

The dashboard features: a large animated AQI number colour-coded by EPA health category; an interactive 72-hour SVG forecast chart that scrubs on hover; three-day summary cards showing min/avg/max AQI; SHAP feature importance bars with animated fills; a health advisory banner; a

model comparison table; and a light/dark theme toggle with a real-time day-cycle sky animation (sun or moon based on Karachi's local time). The design uses two Google Fonts, a particle haze animation on HTML canvas, and CSS variables for theming.

9.2 REST API

A FastAPI application with 9 endpoints is deployed on Hugging Face Spaces via Docker (python:3.11-slim base image). The API provides programmatic access to all prediction data, enabling external systems, mobile apps, monitoring dashboards, alerting tools, to consume forecasts without the Streamlit interface. A 60-minute in-memory cache prevents the full inference pipeline from re-running on every request; if inference fails, the cache returns the last successful result rather than an error.

Endpoint	Description	Response
GET /	HTML landing page with endpoint cards	Instant
GET /health	Model name, RMSE, R ² , cache age	Instant
GET /models	All 4 models with full metrics	Instant
GET /current	Current AQI + weather snapshot	~30s first call
GET /predict	Full 72h + daily summary + SHAP	~30s first call
GET /forecast?hours=N	Configurable hourly predictions	~30s first call
GET /daily	3-day daily summary	~30s first call
GET /shap?top_n=N	Top N feature importances	~30s first call
GET /alert	Current + worst forecast advisory	~30s first call

10. Technical Challenges & Solutions

Twenty-one documented technical challenges were encountered and resolved across the project. The most significant are grouped by category below.

10.1 Data & API Challenges

- Challenge 1, **AQICN station returning stale data**. The default "karachi" city lookup resolved to the US Consulate station, which had been offline since March 2025, returning the same stale AQI=161 value indefinitely. Solution: switched to monitoring station ID A401143 (NED University of Engineering & Technology), identified by iterating through all available Karachi station IDs.

- Challenge 2, **Station returning "-" string**. When the AQICN sensor is temporarily offline it returns a literal dash character instead of a number. `float("-")` raises `ValueError` and crashes the pipeline. Solution: added an explicit guard checking for this case before attempting type conversion.
- Challenge 3, **Open-Meteo archive lag**. The Archive API has a 5-7 day processing delay, requests for recent dates return empty results. Solution: two-API strategy using the Forecast API for the last 7 days and the Archive API for everything older.

10.2 Hopsworks Infrastructure Challenges

- Challenge 4, **Hopsworks version mismatch**. `hopsworks==3.7.0` was incompatible with the backend `v4.7.2` cluster. Required upgrading to `hopsworks[python]==4.7.*` which also changed the API surface for feature group operations.
- Challenge 5, **Feature group schema mismatch**. The first single-row insert created the feature group schema with whatever columns happened to be present, excluding some of the 46 engineered features. Subsequent inserts with the full schema failed with schema mismatch errors. Solution: `enforce_schema()` function added to pad any missing columns with `NaN/0` before every insert.
- Challenge 6, **Offline store Spark job stuck (CPU exhaustion)**. The materialization Spark job entered a permanently stuck state due to Spark CPU exhaustion on the free tier cluster (Kubernetes showed 0/23 nodes available). The offline store was frozen at June 5 data despite the hourly pipeline successfully inserting new rows. Solution: implemented a dynamic fallback in `load_latest_features()` — on each inference run, the function reads the offline store and checks the timestamp of the latest row. If it is more than 1.5 hours old, it automatically switches to the online store (RonDB), which updates instantly on every insert. When the offline store recovers, the next run automatically switches back — no manual intervention required.

10.3 Machine Learning Challenges

- Challenge 7, **Forecast drift (AQI 80 → 283)**. Naive iterative forecasting fed each prediction back as a lag feature, causing small positive biases to compound across 72 steps and produce unrealistically high AQI values. Solution: three-part fix, real history anchor for initial lags, mean reversion force, and sinusoidal diurnal variation.

- Challenge 8, **SHAP returns zero for Ridge**. shap.Explainer (the generic auto-detecting version) gave zero SHAP values for Ridge Regression because it falls back to a method incompatible with linear models. Solution: switched to shap.LinearExplainer which computes exact analytical SHAP values from the Ridge coefficients.
- Challenge 9, **NaN lag features in live rows**. Single-row hourly inserts had no historical context for computing lag features. Solution: fetch last 24 rows from Hopsworks before engineering, prepend, compute features, upload only the new row. Inference pipeline also independently recomputes all lags from actual history.

10.4 Deployment Challenges

- Challenge 10, **Railway and Streamlit Cloud defaulting to Python 3.13**. pandas==2.1.4, tensorflow==2.16.1, and hopsworks all have no pre-built wheels for Python 3.13 and fail to compile from source. Streamlit Cloud was fixed by setting Python 3.11 in Advanced Settings. Railway was abandoned (config files ignored) in favour of Hugging Face Spaces with an explicit python:3.11-slim Docker base image.
- Challenge 11, **Hugging Face build failing, gcc not found**. The hopsworks dependency twofish requires GCC to compile C extensions. The python:3.11-slim image has no build tools. Solution: added apt-get install gcc g++ to the Dockerfile before the pip install step.

11. Known Limitations & Future Work

11.1 Current Limitations

- **Data source mismatch**. The model was trained on Open-Meteo reanalysis AQI values, but live predictions are seeded with AQICN sensor readings. These use different calculation methodologies, so there is a systematic offset between training distribution and live inputs. The model handles this through its strong autocorrelation, recent AQI history dominates the prediction regardless of absolute calibration, but absolute accuracy on live data may differ from the test set metrics.
- **Single monitoring station**. Only one AQICN station (NED UET) is operational in Karachi. If this station goes offline, as the US Consulate station did, there is no live AQI data source. The pipeline handles this gracefully (skips the insert rather than crashing) but predictions would run on stale seed data.

- **Hopsworks free tier CPU.** The offline store materialization job intermittently fails due to Spark CPU exhaustion on the shared free tier cluster. A dynamic fallback is implemented: `load_latest_features()` checks whether the offline store data is more than 1.5 hours stale, and automatically switches to the online store (RonDB) when it is. This resolves itself automatically when the cluster recovers — no manual intervention is needed. The remaining limitation is that the offline store cannot always be used for incremental training on live data during high-load periods.
- **72-hour horizon uncertainty.** Forecast accuracy degrades with horizon. The 1-hour prediction is near-perfect (exploiting $r=0.991$ autocorrelation) but the 72-hour prediction relies increasingly on mean reversion rather than data signal. Users should treat near-term predictions as high-confidence and the 3-day outlook as directional.

13. Conclusion

The Pearls AQI Predictor delivers a complete, working, automated ML system for Karachi's air quality. Every project requirement is satisfied: hourly data ingestion, 13-month historical backfill, four-model training pipeline, Hopsworks feature store and model registry, live dashboard, REST API, EDA notebook, SHAP explainability, and AQI health alerts, all running without manual intervention.

The primary technical insight is that domain understanding matters more than model complexity. EDA revealed that Karachi's AQI exhibits extreme autocorrelation ($r = 0.991$ at 1-hour lag) and approximately linear relationships with all predictors. This made Ridge Regression the structurally correct choice, a conclusion confirmed by the experimental results, where LSTM (the most complex model) scored worst. The data told us the answer before we trained a single model.

The most significant infrastructure challenge, the Hopsworks offline store freezing due to free-tier Spark CPU exhaustion, led to a dynamic architectural solution: `load_latest_features()` now automatically detects whether the offline store is stale (more than 1.5 hours behind) and switches to the online store (RonDB) when needed, reverting to offline when it recovers. This self-healing mechanism is architecturally correct — the online store is designed for exactly this low-latency inference use case — and improved the system rather than simply working around the problem.

Final result: $R^2 = 0.985$, RMSE = 2.35, fully deployed, fully automated. Karachi's next 72 hours of AQI, updated every hour, explained with SHAP, served through a dashboard and REST API, with zero manual intervention required